

Ball Machine

Madina inventou uma máquina de bolas para entreter os participantes da IOI. A estrutura interna da máquina é a seguinte:

- A máquina é constituída por N **nós**, indexados de 0 a $N - 1$. O nó $N - 1$ é designado por **raiz**. Para cada nó u com $0 \leq u < N - 1$, o **pai** do nó u é um nó $P[u]$ com $P[u] > u$, e o nó u é um **filho** de $P[u]$. A raiz não tem pai. Um nó sem filhos é designado por **folha**.

Tu **não** conheces N nem o pai $P[u]$ de qualquer nó u . Em vez disso, Madina informa-te M , o número de folhas, e que os índices das folhas são $0, 1, \dots, M - 1$. A tua tarefa é determinar a estrutura interna da máquina, fazendo operações nela.

Cada nó da máquina pode armazenar no máximo uma **bola**, e cada bola tem um **valor** que é um número inteiro não negativo. Dizemos que um nó tem valor x se armazenar uma bola com valor x . Um nó está **ocupado** se contiver uma bola; caso contrário, está **livre**. Inicialmente, todos os nós estão livres.

Podes realizar dois tipos de operações:

- insert** (U, X): tentativa de inserir uma nova bola com o valor X na folha U .
 - Se a folha U estiver ocupada, a operação devolve `false` sem inserir a bola.
 - Se a folha U estiver livre, a bola é colocada no nó U . De seguida, a bola move-se repetidamente para o pai do seu nó atual, desde que esse pai esteja livre. O movimento pára quando a bola atinge a raiz ou um nó cujo pai está ocupado. A operação retorna `true`.
- collect()**: se a raiz estiver livre (o que significa que a máquina está vazia), `collect` devolve um array vazio. Caso contrário, a função recursiva `traverse` descrita abaixo recolhe os valores de todas as bolas que estão actualmente na máquina num array S . Inicialmente, o array S está vazio e é chamado `traverse(N - 1)`.

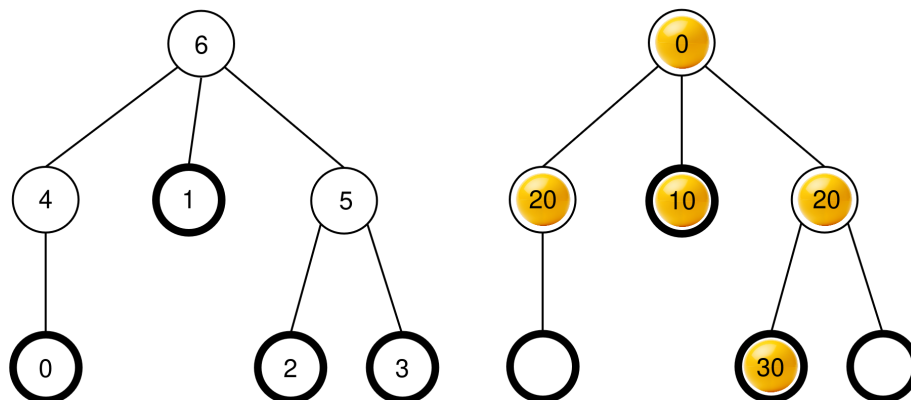
```

traverse(u) :
  Adicionar o valor da bola no nó u no final de S.
  Seja c[u] a lista dos filhos ocupados de u
  Ordenar c[u] por ordem não decrescente de acordo com os
    valores das bolas nesses nós (se vários nós tiverem o
    mesmo valor, podem aparecer por qualquer ordem)
  para cada v em c[u]:
    traverse(v)

```

Após a conclusão desta função, **todas as bolas são removidas** da máquina e `collect` retorna S .

Por exemplo, considera uma máquina com $N = 7$ nós e um array pai $P = [4, 6, 5, 5, 6, 6]$. A imagem da esquerda mostra a máquina vazia com os índices dos nós, enquanto a imagem da direita mostra uma possível configuração de bolas após uma sequência de operações de `insert` (esta sequência é fornecida na secção Exemplo).



Quando `collect()` é chamado, a raiz (nó 6) está ocupada, pelo que S é inicializado como $S = []$ e `traverse(6)` é chamado.

traverse(6) : o nó 6 tem o valor 0; adicionando-o a S resulta em $S = [0]$. Os filhos do nó 6 são 4, 1 e 5, todos ocupados. Os seus valores são 20, 10 e 20, respetivamente. Ordenando por ordem não decrescente, obtemos $c[6] = [1, 5, 4]$ (note-se que $c[6] = [1, 4, 5]$ também seria válido, uma vez que os nós 4 e 5 têm o mesmo valor). A função chama então `traverse` em cada nó de $c[6]$ por esta ordem.

- **traverse(1)** : o nó 1 tem o valor 10; adicionando-o a S resulta em $S = [0, 10]$. O nó 1 não tem filhos ocupados, pelo que esta chamada termina.
- **traverse(5)** : o nó 5 tem o valor 20; adicionando-o a S resulta em $S = [0, 10, 20]$. O nó 5 tem um filho ocupado, o nó 2, logo $c[5] = [2]$ e a função chama `traverse(2)`.
 - **traverse(2)** : o nó 2 tem o valor 30; adicionando-o a S resulta em $S = [0, 10, 20, 30]$. O nó 2 não tem filhos ocupados, pelo que esta chamada termina.

- A execução retorna para `traverse(5)` que processou todos os filhos em $c[5]$, pelo que a sua chamada termina.
- **`traverse(4)`** : o nó 4 tem o valor 20; adicionando-o a S resulta em $S = [0, 10, 20, 30, 20]$. O nó 4 não tem filhos ocupados, pelo que esta chamada termina.

Todas as chamadas foram concluídas e não existem mais nós para processar. Por fim, todas as bolas são retiradas da máquina e `collect` devolve o array $S = [0, 10, 20, 30, 20]$.

A tua tarefa é determinar o número de nós N e reportar um vetor de pais $R = [R[0], R[1], \dots, R[N-2]]$ que descreva a estrutura da máquina, mas com uma rotulagem potencialmente diferente para os nós que não são nem folhas nem a raiz. Formalmente, um vetor reportado R é considerado correto se for possível atribuir um rótulo distinto $L[u]$ com $0 \leq L[u] < N$ a cada nó u da máquina, de modo a que:

- $L[u] = u$ para todo $0 \leq u < M$ e $u = N-1$, e
- $L[P[u]] = R[L[u]]$ para todo $0 \leq u < N-1$.

Não é necessário que $R[u] > u$. A máquina **deve estar vazia** quando reportas a tua solução.

Seja K o número de operações `collect` realizadas e B o valor máximo de qualquer bola em todas as chamadas `insert`. Seja $C = K + B$. Então, **o valor C não deve exceder 1000**. A tua pontuação em algumas subtarefas depende do valor de C .

Detalhes da Implementação

Deves implementar a seguinte função:

```
std::vector<int> find_structure(int M)
```

- M : o número de folhas na máquina.
- A função é chamado exactamente uma vez para cada caso de teste.
- A função deve devolver um array $R = [R[0], R[1], \dots, R[N-2]]$ de comprimento $N-1$, um array correcto de pais.

Para interagir com a máquina, a tua função pode chamar as duas funções seguintes.

A primeira função é:

```
bool insert(int U, int X)
```

- U : o índice da folha onde a bola deve ser inserida. A condição $0 \leq U < M$ deve ser satisfeita.
- X : o valor inteiro da bola. A condição $0 \leq X \leq 1000$ deve ser satisfeita.

- Esta função retorna `true` se a bola foi introduzida com sucesso, ou `false` se a folha U já estava ocupada.
- Esta função pode ser chamada no máximo 500 000 vezes.

A segunda função é:

```
std::vector<int> collect()
```

- Recolhe os valores de todas as bolas introduzidas, esvazia a máquina e devolve o array recolhido S .

Se em algum momento durante a execução do teu programa o valor de C exceder 1000, a tua solução receberá o veredicto `Output isn't correct: Too many resources used`.

A estrutura da máquina é definida antes da chamada de `find_structure`. O avaliador é **determinístico** no sentido em que se o executares duas vezes e ambas as execuções realizarem a mesma sequência de operações, as chamadas a `collect` devolverão os mesmos arrays.

Restrições

- $2 \leq N \leq 1000$
- $1 \leq M \leq 200$
- $M < N$

Subtarefas

Subtarefa	Pontos	Restrições Adicionais
1	5	A raiz tem exatamente M filhos.
2	10	$M \leq 3$
3	25	$N \leq 200, M \leq 45$
4	60	Sem restrições adicionais.

Nas subtarefas 3 e 4, a tua pontuação depende do valor de C da seguinte forma.

Subtarefa 3 (25 pontos)

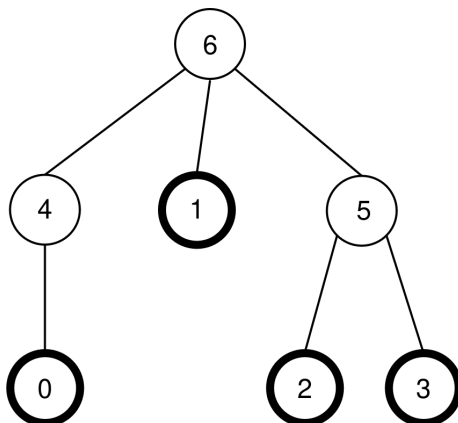
Limites	Pontuação
$1000 < C$	0
$45 < C \leq 1000$	13
$C \leq 45$	25

Subtarefa 4 (60 pontos)

Limites	Pontuação
$1000 < C$	0
$200 < C \leq 1000$	7
$71 < C \leq 200$	$47 - \frac{C}{5}$
$44 < C \leq 71$	$104 - C$
$C \leq 44$	60

Exemplo

Considera a mesma máquina da descrição da tarefa, pelo que $N = 7$, $M = 4$ e $P = [4, 6, 5, 5, 6, 6]$. A máquina é novamente mostrada na figura seguinte:



O avaliador faz a seguinte chamada:

```
find_structure(4)
```

A função pode realizar a seguinte sequência de chamadas:

1. **insert(0, 0)** : a folha 0 está livre, pelo que é colocada uma bola com o valor 0 na folha 0. O nó $P[0] = 4$ está livre, pelo que a bola se desloca para o nó 4. O nó $P[4] = 6$ está livre, pelo que a bola se desloca para o nó 6. Como o nó 6 é a raiz, o movimento pára. A chamada retorna `true`.
2. **insert(3, 20)** : a folha 3 está livre, pelo que é colocada uma bola com o valor 20 na folha 3. O nó $P[3] = 5$ está livre, pelo que a bola se desloca para o nó 5. Como $P[5] = 6$ está ocupado, o movimento pára. A chamada retorna `true`.
3. **insert(1, 10)** : a folha 1 está livre, pelo que é colocada uma bola com o valor 10 na folha 1. Como $P[1] = 6$ está ocupado, a bola permanece no nó 1. A chamada retorna `true`.

4. `insert(2, 30)` : a folha 2 está livre, pelo que é colocada uma bola com o valor 30 na folha 2. Como $P[2] = 5$ está ocupado, a bola permanece no nó 2 . A chamada retorna `true` .
5. `insert(1, 25)` : a folha 1 está ocupada, pelo que a bola não é colocada na folha 1. A chamada retorna `false` .
6. `insert(0, 20)` : a folha 0 está livre, pelo que é colocada uma bola com o valor 20 na folha 0. O nó $P[0] = 4$ está livre, pelo que a bola se desloca para o nó 4 . Como $P[4] = 6$ está ocupado, o movimento pára. A chamada retorna `true`.

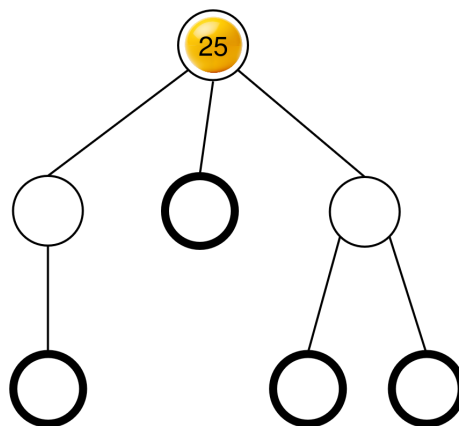
A configuração resultante das bolas já tinha sido apresentada na descrição do problema.

Em seguida, a função pode chamar:

```
collect()
```

A execução desta chamada já foi explicada na descrição do problema, pelo que esta chamada retorna $S = [0, 10, 20, 30, 20]$. Depois disso, a máquina fica novamente vazia.

De seguida, a função pode chamar `insert(2, 25)` . A folha 2 está livre, pelo que é colocada uma bola com o valor 25 na folha 2. Esta bola move-se então para o nó 5 e depois para o nó 6, onde pára. A chamada retorna `true`. A configuração resultante das bolas é apresentada na figura seguinte.



Por fim, a função pode chamar `collect()` que retorna $S = [25]$ e esvazia a máquina.

Existem dois arrays pais correctos que `find_structure` pode devolver: $R = P = [4, 6, 5, 5, 6, 6]$ (que corresponde à rotulagem $L = [0, 1, 2, 3, 4, 5, 6]$), e $R = [5, 6, 4, 4, 6, 6]$ (que corresponde à rotulagem $L = [0, 1, 2, 3, 5, 4, 6]$). Neste exemplo, $K = 2$ e $B = 30$, logo $C = 32$.

Avaliador de Exemplo

Formato do Input:

```
N M
P[0] P[1] P[2] ... P[N-2]
```

Formato do Output:

```
K B C
Q
R[0] R[1] R[2] ... R[Q-1]
```

Aqui, Q é o comprimento da matriz R devolvida por `find_structure`.